

Curso desarrollo de aplicaciones con tecnologías web
Módulo formativo 2: Programación web en el entorno servidor

Unidad formativa 2: Acceso a datos en aplicaciones web del entorno servidor

Tema 1: Modelos de Datos

Concepto de Dato y Ciclo de Vida de los Datos

Tipos de Datos

Definición de un Modelo Conceptual

El Modelo Relacional

Concepto de Dato y Ciclo de Vida de los Datos:

¿Qué es un dato?

Un dato es una representación simbólica de una información, puede ser un número, texto, imagen o cualquier otro valor que tenga significado en un contexto determinado.

Ciclo de vida de los datos

1. **Creación:** Los datos son generados o capturados desde una fuente de entrada (formularios, sensores, APIs, etc.).
2. **Almacenamiento:** Se guardan en bases de datos o sistemas de archivos.
3. **Procesamiento:** Se analizan, modifican o combinan para extraer información útil.
4. **Distribución:** Se presentan o envían a otros sistemas o usuarios.
5. **Archivado o eliminación:** Los datos se guardan a largo plazo o se eliminan cuando ya no son necesarios.

Ejemplo: En un sistema de comercio electrónico, un "dato" sería el nombre de un usuario o el precio de un producto.

Tipos de Datos

1. Datos Básicos

Son valores simples que no están estructurados en conjuntos complejos.

- **Numéricos:** Enteros (`int`), decimales (`float`).
- **Texto:** Cadenas de caracteres (`string`).
- **Booleanos:** Verdadero o falso (`true / false`).

Ejemplo en PHP:

```
$nombre = "Juan"; // String  
$edad = 25; // Entero  
$precio = 19.99; // Decimal  
$activo = true; // Booleano
```

2. Registros

Conjuntos de datos organizados en una estructura fija.

- En bases de datos, un registro es una fila en una tabla.

Ejemplo de registro en SQL:

```
INSERT INTO usuarios (id, nombre, email) VALUES (1, 'Juan Pérez', 'juan@example.com');
```

3. Datos Dinámicos

Pueden cambiar en tiempo de ejecución y depender de la interacción del usuario o el sistema.

- Listas (arrays en PHP).
- Objetos (clases en programación orientada a objetos).

Ejemplo en PHP:

```
$productos = ["Laptop", "Mouse", "Teclado"];  
echo $productos[0]; // Laptop
```

Definición de un Modelo Conceptual

Un **modelo conceptual** representa la estructura lógica de los datos sin detalles técnicos. Se usa para definir cómo se relacionan los datos.

1. Patrones

Los patrones de modelado establecen reglas comunes para organizar la información.

- **Modelo entidad-relación:** Describe entidades, atributos y relaciones.
- **Modelo orientado a objetos:** Usa clases y objetos para representar datos.

2. Modelos Genéricos

- **Jerárquico:** Organiza los datos en forma de árbol.
- **Red:** Permite relaciones más complejas.
- **Relacional:** Usa tablas con filas y columnas.

El Modelo Relacional

1. Descripción

El modelo relacional representa datos en **tablas** con filas y columnas. Cada fila es un registro y cada columna un atributo.

2. Entidades y Tipos de Entidades

- **Entidad:** Un objeto del mundo real con atributos (ej. Cliente, Producto).
- **Entidad fuerte:** Existe por sí misma (ej. Cliente).
- **Entidad débil:** Depende de otra (ej. Detalle de un pedido).

3. Elementos de Datos. Atributos

- **Atributos simples:** Un solo valor (ej. nombre).
- **Atributos compuestos:** Se pueden dividir en subpartes (ej. nombre_completo).
- **Atributos clave:** Identifican unívocamente un registro (ej. id_usuario).

4. Relaciones. Tipos, Subtipos, Cardinalidad

Las relaciones establecen vínculos entre entidades:

- **Uno a uno (1:1):** Un usuario tiene un perfil.
- **Uno a muchos (1:N):** Un cliente puede hacer varios pedidos.
- **Muchos a muchos (M:N):** Un producto puede pertenecer a varias categorías.

Ejemplo de relación en SQL:

```
CREATE TABLE pedidos (  
    id INT PRIMARY KEY,  
    id_cliente INT,  
    FOREIGN KEY (id_cliente) REFERENCES clientes(id)  
);
```

5. Claves. Tipos de Claves

- **Clave primaria (PK):** Identifica un registro de manera única (`id`).
- **Clave foránea (FK):** Enlace entre tablas (`id_cliente` en `pedidos`).

6. Normalización. Formas Normales

Proceso para reducir la redundancia y mejorar la integridad de los datos.

- **1ª Forma Normal (1NF):** Evitar datos repetidos en una celda.
- **2ª Forma Normal (2NF):** Separar datos en tablas relacionadas.
- **3ª Forma Normal (3NF):** Eliminar dependencias transitivas.

Ejemplo de mejora de diseño: Antes (sin normalización):

```
CREATE TABLE pedidos (  
    id INT PRIMARY KEY,  
    cliente VARCHAR(255),  
    direccion VARCHAR(255)  
);
```

Después (con normalización en 2NF):

```
CREATE TABLE pedidos (  
    id INT PRIMARY KEY,  
    id_cliente INT,  
    FOREIGN KEY (id_cliente) REFERENCES clientes(id)  
);
```

Uso de JSON en Bases de Datos Modernas

Las bases de datos modernas permiten almacenar información en formato **JSON** para mayor flexibilidad. JSON es un formato de datos ligero y estructurado que facilita la manipulación de información sin necesidad de crear múltiples tablas relacionadas. Esto es especialmente útil para **almacenar listas de productos en pedidos, configuraciones personalizadas o datos semiestructurados**.

Por ejemplo, en lugar de tener una tabla intermedia para los productos de un pedido, podemos almacenarlos directamente en una columna JSON:

```
CREATE TABLE pedidos (  
  id INT PRIMARY KEY,  
  id_cliente INT,  
  productos JSON,  
  fecha_pedido DATETIME DEFAULT CURRENT_TIMESTAMP  
);
```

Esto permite almacenar en productos un array JSON como:

```
[  
  {"id": 1, "nombre": "Laptop", "precio": 799.99},  
  {"id": 2, "nombre": "Mouse", "precio": 19.99}  
]
```

Ventajas de usar JSON en bases de datos:

- **Flexibilidad:** No es necesario modificar el esquema de la base de datos para agregar nuevos atributos.
- **Menos JOINs:** Reduce la necesidad de crear múltiples tablas relacionadas.
- **Mayor velocidad de lectura** en ciertos casos donde las consultas son más simples.

Desventaja: JSON no es eficiente para búsquedas complejas con SQL puro, por lo que es mejor usarlo en combinación con índices adecuados.

Ejemplo en PHP con PDO

A continuación, se muestra un ejemplo de cómo consultar productos desde una base de datos MySQL usando PDO en PHP:

```
$conexion = new PDO("mysql:host=localhost;dbname=curso_web", "root", "");  
$stmt = $conexion->prepare("SELECT * FROM productos WHERE categoria_id = :id_categoria");  
$stmt->execute(['id_categoria' => 1]);  
while ($producto = $stmt->fetch(PDO::FETCH_ASSOC)) {  
    echo $producto['nombre'] . "<br>";  
}
```

Explicación del código:

- Se crea una conexión a MySQL usando PDO.
- Se prepara una consulta SQL con `:id_categoria` como parámetro.
- Se ejecuta la consulta pasando el valor correspondiente.
- Se recorren los resultados con `fetch()` y se muestran los nombres de los productos.

Beneficio de usar PDO: Permite consultas seguras y preparadas, evitando inyecciones SQL.

Ejercicio Práctico (Opcional)

1. Crear un diagrama entidad-relación con **Cientes, Pedidos y Productos**.
2. Diseñar las tablas en SQL usando claves primarias y foráneas.
3. Aplicar la normalización hasta la **2ª Forma Normal**.